Algorithms

Mihir Rao

Based on Algorithm Design by Kleinberg & Tardos

Contents

1 Introduction: Some Representative Problems **2**
- 1.1 Stable matching: the model . . . 2
- 1.2 Examples: stability need not be unique . . . 3
- 1.3 Gale–Shapley: tentative commitments . . . 3
- 1.4 Termination, perfection, and stability . . . 4
- 1.5 Which stable matching does the algorithm choose? . . . 5
- 1.6 Five representative problems . . . 6
- 1.7 Worked extensions of stable matching . . . 9

Chapter 1: Introduction: Some Representative Problems

1.1 Stable matching: the model

An algorithmic problem starts with a precise model of what a solution must do. For matching applicants to employers, merely assigning everyone a place does not make an outcome self-enforcing: an applicant and an employer might both prefer one another to their assigned partners. Stability rules out such a pair.

Use the book's two sets

$$M = \{m_1, \ldots, m_n\}, \qquad W = \{w_1, \ldots, w_n\}.$$

Each member of either set ranks *every* member of the other set in a strict preference list, with no ties. The names men and women describe the two sides of this abstract model. Equal set sizes, complete lists, and one-to-one pairing are assumptions of the basic problem.

Definition 1.1 [Matching and stability]. A matching $S \subseteq M \times W$ uses each individual in at most one pair. It is perfect if each individual appears in exactly one pair. For a perfect matching S, an instability is a pair $(m, w') \notin S$ such that, writing $(m, w), (m', w') \in S$, both of the following hold:

- m prefers w' to w;
- w' prefers m to m'.

A matching is stable if it is perfect and has no instability.

One dissatisfied individual is insufficient: a deviation needs two people who both prefer it. Stability is a condition on all unmatched pairs, not a claim that everyone receives a first choice.

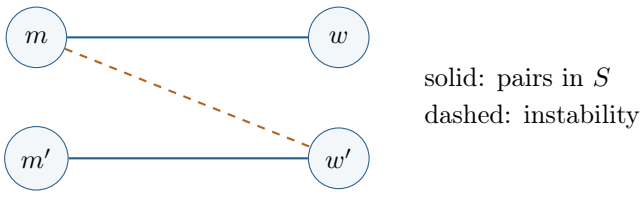


Figure 1: The unmatched pair (m, w') blocks the current assignment when each prefers the other to the partner on the solid edge.

1.2 Examples: stability need not be unique

With two people on each side, suppose both men rank w above w' and both women rank m above m'. Then $\{(m, w), (m', w')\}$ is the unique stable matching: the other perfect matching is blocked by (m, w).

Now let the preferences be

m	w before w'	w	m' before m
m'	w' before w	w'	m before m'.

Here both perfect matchings are stable. In $\{(m, w), (m', w')\}$, both men have their first choices, so neither can be part of an instability. In $\{(m, w'), (m', w)\}$, both women have their first choices. Thus a procedure can be correct without being able to produce every stable matching.

1.3 Gale–Shapley: tentative commitments

An accepted proposal cannot immediately be final: a woman may later receive a proposal from someone she prefers. Rejecting every proposal while waiting for a better one is also unsafe, since that better proposal may never arrive. The solution is a tentative ENGAGEMENT. Each woman keeps the best proposal she has received so far; each free man continues down his list.

Algorithm 1.1: Gale–Shapley, with men proposing

```
Input: Complete, strict preference lists for M and W, with |M| = |W| = n
Mark every person free
while some man m is free and has not proposed to every woman do
    Choose such a man m
    Let w be the highest-ranked woman on m's list not yet proposed to
    Record that m has proposed to w
    if w is free then
        Engage m and w
    else
        Let m' be w's current partner
        if w prefers m to m' then
            Engage m and w; make m' free
        else
            Leave m free
return the set S of engaged pairs
```

The choice of the next free man is arbitrary. Engagements always form a matching: a free man proposes, and any replaced partner becomes free. A rejected man never proposes to the same woman again.

Lemma 1.2 [Monotonicity]. *Once a woman receives a proposal, she remains engaged, and her partner can only improve in her preference order. Each man's successive proposals move strictly downward through his preference list.*

Proof. A woman accepts her first proposal and subsequently changes partner only for someone she prefers. A man always selects his best remaining untried option. Neither rule requires the overall number of engagements to increase. □

1.4 Termination, perfection, and stability

These are three different claims. A terminating algorithm might leave someone unmatched; a perfect matching might still contain an instability.

Proposition 1.3 [At most n^2 proposals]. *The while loop executes at most n^2 times.*

Proof. Let $P(t)$ be the set of pairs (m, w) for which m has proposed to w by the end of iteration t. Every iteration adds one previously absent pair, so

$$|P(t+1)| = |P(t)| + 1, \qquad P(t) \subseteq M \times W, \qquad |P(t)| \leq n^2.$$

This is a progress measure that cannot increase forever. The number of free people would not work: replacing a woman's partner frees one man while engaging another, leaving that number unchanged. □

This bounds iterations. Turning it into a running-time bound also requires controlling the cost of choosing a free man, locating his next choice, and comparing ranks; the data-structure implementation is developed in Chapter 2.

Lemma 1.4 [A free man has an untried woman]. *At every point in the basic algorithm, a free man has not yet proposed to all n women.*

Proof. If a free man m had proposed to all women, every woman would have received a proposal and would still be engaged. Their distinct partners would be n engaged men, but only $n - 1$ men other than m are available. This is impossible. □

Theorem 1.5 [Existence and correctness]. *Gale–Shapley returns a stable matching for every instance of the basic model.*

Proof. Termination follows from the proposal bound. If a man were free at termination, the previous lemma would give him an untried woman, so the loop could still run. Hence everyone is matched and S is perfect.
Suppose (m, w') were an instability, with $(m, w), (m', w') \in S$. Since m prefers w' to w, he proposed to w' before reaching w. He was either rejected immediately or accepted and later displaced. At that rejection, w' kept a man she preferred to m. Her partner could only improve afterward, so her final partner m' is also preferred to m. This contradicts the other condition for an instability. □

The core mechanism is asymmetric progress: men move down their lists while women's retained partners improve. Rejection is therefore enough evidence to exclude a blocking pair at the end, even though it does not finalize everyone else's assignment immediately.

1.5 Which stable matching does the algorithm choose?

The two-person example shows that men proposing can favor the men. In fact, the preference guarantee holds for *all* men simultaneously, and the arbitrary order of proposals cannot change the answer.

Definition 1.6 [Valid partners]. A woman w is a valid partner of m if some stable matching contains (m, w). The best valid partner, denoted $\text{best}(m)$, is m's highest-ranked valid partner. Define

$$S^* = \{(m, \text{best}(m)) : m \in M\}.$$

A best valid partner need not be a first choice on the original preference list. Validity asks what is attainable while preserving stability for everyone. At this stage, even the assertion that S^* is a matching needs proof: two men might appear to have the same best valid partner.

Theorem 1.7 [Proposer optimality and order independence]. *Every execution of men-proposing Gale–Shapley returns S^*.*

Proof. Assume an execution rejects a man from some valid partner. Choose the *first* such rejection: w rejects m in favor of m', whom she prefers. This includes both immediate rejection and breaking an earlier engagement. Because w is valid for m, there is a stable matching S' containing (m, w). Let $(m', w') \in S'$, where $w' \neq w$.
The woman w' is a valid partner of m'. When m' proposed to w, no man had yet been rejected by a valid partner. If m' preferred w' to w, he would already have proposed to w' and been

rejected before he could propose to w. That would be an earlier rejection by a valid partner, contradicting our choice. Therefore m' prefers w to w'.
Now (m', w) blocks S': m' prefers w to his partner w', and w prefers m' to her partner m. This contradicts stability. Thus no man is ever rejected by a valid partner. Since proposals follow preference order, and the returned matching is stable, each man must end with his best valid partner. □

The conclusion proves that S^* really is a stable matching and that all executions agree. It does not imply that the instance has only one stable matching. Reversing the proposing side may give a different result.

Proposition 1.8 [The receiving side's outcome]. *In S^*, every woman receives her worst valid partner.*

Proof. Suppose $(m, w) \in S^*$ but some stable matching S' pairs w with m', whom she likes less than m. In S', let m be paired with $w' \neq w$. Because $w = \text{best}(m)$ and w' is valid for m, he prefers w to w'. Thus m and w prefer each other to their partners in S', a contradiction. □

The word *valid* is essential: this is the worst partner attainable across stable matchings, not necessarily the last person on a woman's full list.

1.6 Five representative problems

A graph $G = (V, E)$ represents pairwise relationships. Its elements of V are nodes; an edge $e = \{u, v\} \in E$ joins two nodes, its ends. The next five problems show how modest changes to a model alter the design techniques and difficulty.

1.6.1 Interval scheduling

There are n requests. Request i uses one resource from s_i to f_i, with $s_i < f_i$. Two requests i, j are compatible precisely when

$$f_i \leq s_j \quad \text{or} \quad f_j \leq s_i.$$

A finish can coincide with the next start. A subset A is compatible when every pair of its requests is compatible. The objective is

$$\max\{|A| : A \text{ is compatible}\}.$$

The problem admits a greedy algorithm: order requests by a suitable rule and process them with irrevocable choices. The significance is that a local rule can achieve a global optimum; the correct rule and proof come later.

1.6.2 Weighted interval scheduling

Now each request has a value $v_i > 0$, and the objective changes to

$$\max \left\{ \sum_{i \in A} v_i : A \text{ is compatible} \right\}.$$

Setting all values to one recovers interval scheduling. However, a high-value request can be worth more than all the other requests combined, in which case an optimum must include it, regardless of how many small requests it excludes. A rule concerned only with the number of intervals loses this information. Dynamic programming will compare alternatives through a compact table of optimal subproblem values.

1.6.3 Bipartite matching

A graph is BIPARTITE when V partitions into two sets X, Y and every edge joins X to Y. A matching in a graph is a set of edges $M \subseteq E$ with no shared ends. Here M denotes an edge set, whereas it denoted the set of men in stable matching. A perfect matching covers every node exactly once.

The objective is to maximize $|M|$. If $|X| = |Y| = n$, a perfect matching exists exactly when the maximum size is n. Edges might represent a machine's ability to run a job, or a professor's ability to teach a course. Some pairs are unavailable, and there are no preference lists in this problem. The constraint is feasibility of assignments, not absence of mutually preferred deviations.

An efficient solution grows a matching through AUGMENTATION, revising some earlier choices to make room for more edges. This leads to network flow; simple irrevocable assignment does not capture the required flexibility.

1.6.4 Independent set and conflict graphs

A set $S \subseteq V$ is INDEPENDENT if no edge has both ends in S. The objective is to maximize $|S|$. An edge now means that two candidate objects conflict, so they cannot both be selected.

Interval scheduling becomes independent set by making one node per interval and joining overlapping intervals. A compatible set of requests is exactly an independent set in this graph. For matching, the objects to select are *edges*, so the transformation uses a different level of representation:

- Start with the bipartite graph $G' = (V', E')$.
- Form a conflict graph G whose node set is $V = E'$.
- Join two nodes of G when the corresponding edges of G' share an end.

An independent set of G is then exactly a matching of G', with the same size. The transformation preserves the objective because each chosen node represents one chosen edge.

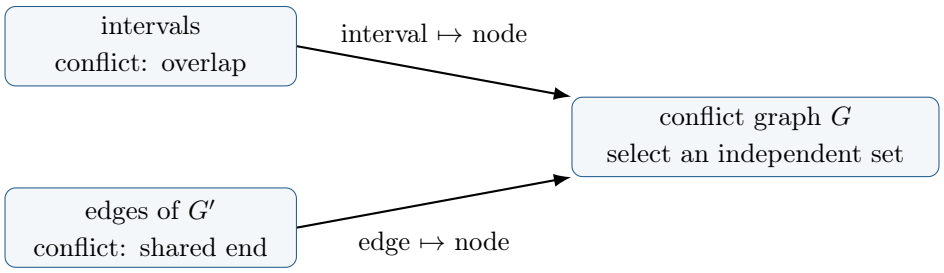


Figure 2: The same conflict-graph model expresses two different selection problems. The resulting graphs have special structure; arbitrary graphs allow more general conflict patterns.

For general graphs, no efficient exact algorithm is known. Brute force tries all $2^{|V|}$ node subsets. The decision version, asking whether an independent set of size at least a target exists, is NP-complete. This does not prove that an efficient algorithm is impossible. It identifies a family of problems for which such an algorithm would have far-reaching consequences.

There is still an easy verification task: given a proposed set of the required size, check that none of its pairs is an edge. This certifies that a sufficiently large independent set exists; it does *not* certify that the given set is maximum. Finding a solution and checking a supplied witness are distinct tasks.

1.6.5 Competitive facility location

Zones $1, \ldots, n$ have values b_i, and an edge represents a pair of zones in which two franchises cannot coexist. Players P_1 and P_2 alternately select nodes, with P_1 moving first. No node can be selected twice, and all selected nodes together must remain an independent set. A selected zone contributes its value to the player selecting it.

Given a target B for P_2, ask whether P_2 has a strategy guaranteeing a total value of at least B, regardless of how P_1 plays. A strategy must specify responses to different opponent moves; it is more than one successful sequence of play. For the weighted path below, adjacent zones conflict.

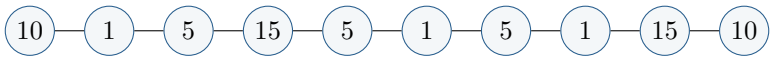


Figure 3: Node labels are zone values b_i. Selecting a zone prevents either player from later selecting either neighboring zone.

P_2 can guarantee 20 but cannot guarantee 25.

The problem illustrates PSPACE-completeness. In contrast to displaying an independent set, establishing a winning strategy can require reasoning through many possible opposing choices. PSPACE-complete problems are believed to be harder than NP-complete problems; this separation

is a conjecture, not a proven strict hierarchy. The chapter uses these problems to distinguish efficient construction, easy verification of witnesses, and strategic case analysis.

1.7 Worked extensions of stable matching

1.7.1 A commonly preferred group

Suppose there are k good men and k good women, with $1 \le k \le n-1$, and everyone ranks all good people on the opposite side above all bad people. Every stable matching pairs good men with good women.

Proof. If a good man m were matched to a bad woman, at most $k-1$ good men would remain available for the k good women. Some good woman w' would therefore be paired with a bad man. Both m and w' prefer any good partner to their current bad partners, so (m, w') is an instability. Counting finds the second person needed for the contradiction. □

1.7.2 Forbidden pairs and incomplete lists

Let $F \subseteq M \times W$ be a set of forbidden pairs. Each person ranks only nonforbidden partners, and a matching may use no pair from F. Being single is preferable to a forbidden pairing, but every listed partner is acceptable. A stable matching need no longer be perfect.

Stability now excludes four kinds of deviation involving an allowed pair:

1. Two matched people each prefer the other to their current partner.
2. A free man and a matched woman would prefer to pair with one another.
3. A matched man and a free woman would prefer to pair with one another.
4. A free man and a free woman could pair with one another.

Run the same proposal procedure on the shortened lists. The loop continues only while a free man has an untried *nonforbidden* woman. A free man may now exhaust his list, which is why the perfect-matching proof no longer applies. Each allowed pair receives at most one proposal, so termination still follows, with at most n^2 iterations. Women still remain engaged after their first proposal and only improve their partners.

Proposition 1.9 [Stability with forbidden pairs]. *The modified procedure always returns a stable matching under this expanded definition.*

Proof. A man who finishes free has proposed to every allowed woman; a woman who finishes free has received no proposal. Check the four possible instabilities.
For type 1, a man preferring an allowed woman to his final partner must already have proposed to her. She rejected him and finishes with someone she prefers. For type 2, the free man also must have proposed to that woman, so the same rejection argument applies.
For type 3, a man would have proposed to the preferred free woman before his final partner. But a woman who receives a proposal cannot finish free. For type 4, the free man has exhausted his allowed list, so he must have proposed to the free woman, again impossible. No allowed blocking pair remains. □

Source. Jon Kleinberg and Éva Tardos, Algorithm Design, first edition, Chapter 1, pp. 1–28 (PDF pp. 27–54).